ECS 174 Computer Vision

Homework 1

Adam Hetherwick

October 2026

This code is meant to be run on Google Colab for accurate file loading and GPU usage.

Instructions: For all the following sections, train the model for 50 epochs and plot the curve for loss, training accuracy, and test accuracy evaluated every epoch. In scenarios where training takes so long, you may decrease total number of epochs.

Problem 1

Run the tutorial code out of the box and make sure you get reasonable results. You will report these results in Section 4, so no report needed here. https://pytorch.org/tutorials/_downloads/cifar10_tutorial.py

```
# Package import

import torch
import torchvision
import torchvision.transforms as transforms
```

```
transform = transforms.Compose(
    [transforms.ToTensor(),                                     # .ToTensor() rearranges dimension to channels x height x width a
     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])   # .Normalize() scales to [-1, 1]
```

```
from datasets import load_dataset
from torch.utils.data import Dataset
```

```
# Import data from local machine to avoid constant redownloading

from google.colab import drive
drive.mount('/content/drive')

import os, shutil

src = '/content/drive/MyDrive/cifar_cache/cifar-10-python.tar.gz'
os.makedirs('./data', exist_ok=True)
shutil.copytree('/content/drive/MyDrive/cifar-10-batches-py',
                './data/cifar-10-batches-py')
```

```
Mounted at /content/drive
'./data/cifar-10-batches-py'
```

```
trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                        download=False, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=128,
                                          shuffle=True, num_workers=2)

testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=False, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=128,
                                         shuffle=False, num_workers=2)
```

```
classes = ('plane', 'car', 'bird', 'cat',
           'deer', 'dog', 'frog', 'horse', 'ship', 'truck')
```

```
# Visualize images

import matplotlib.pyplot as plt
import numpy as np
import random
```

```
def imshow(img):
    img = img / 2 + 0.5     # unnormalize
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))

i = random.randint(0, len(trainset) - 1)
img, label = trainset[i]
imshow(img)
plt.title(classes[label])
plt.axis('off')
plt.show()
```




```
import torch.nn as nn
import torch.nn.functional as F
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(device)
```

```
cuda
```

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 16 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

net = Net()
```

```
# Put model on GPU
net = Net().to(device)
```

```
import torch.optim as optim

criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)
```

```
def evaluate(model, loader):
    """Return accuracy (%) of the model on a dataloader."""
    model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            preds = model(images).argmax(dim=1)
            correct += (preds == labels).sum().item()
            total += labels.size(0)
    return 100 * correct / total
```

```
from tqdm import tqdm

num_epochs = 50
train_losses, train_accs, test_accs = [], [], []

for epoch in range(num_epochs):
    net.train()
    running_loss, correct, total = 0.0, 0, 0

    for inputs, labels in tqdm(trainloader, desc=f'Epoch {epoch + 1}/{num_epochs}', leave=False):
        inputs, labels = inputs.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        correct += (outputs.argmax(dim=1) == labels).sum().item()
        total += labels.size(0)

    train_loss = running_loss / len(trainloader)
    train_acc = 100 * correct / total
    test_acc = evaluate(net, testloader)

    train_losses.append(train_loss)
    train_accs.append(train_acc)
    test_accs.append(test_acc)

    print(f'Epoch {epoch + 1}/{num_epochs} | loss: {train_loss:.4f} | '
          f'train acc: {train_acc:.2f}% | test acc: {test_acc:.2f}%')

print('Finished Training')
```

```
Epoch 1/50 | loss: 2.3018 | train acc: 10.43% | test acc: 12.52%
Epoch 2/50 | loss: 2.2868 | train acc: 15.52% | test acc: 15.47%
Epoch 3/50 | loss: 2.1880 | train acc: 20.38% | test acc: 25.51%
Epoch 4/50 | loss: 2.0445 | train acc: 27.60% | test acc: 30.01%
Epoch 5/50 | loss: 1.8889 | train acc: 31.64% | test acc: 33.17%
Epoch 6/50 | loss: 1.7828 | train acc: 34.72% | test acc: 36.28%
Epoch 7/50 | loss: 1.7182 | train acc: 37.00% | test acc: 37.47%
Epoch 8/50 | loss: 1.6620 | train acc: 39.15% | test acc: 38.93%
Epoch 9/50 | loss: 1.6087 | train acc: 41.23% | test acc: 42.26%
Epoch 10/50 | loss: 1.5652 | train acc: 42.86% | test acc: 43.11%
Epoch 11/50 | loss: 1.5296 | train acc: 44.48% | test acc: 44.98%
Epoch 12/50 | loss: 1.4943 | train acc: 45.73% | test acc: 45.70%
Epoch 13/50 | loss: 1.4631 | train acc: 46.91% | test acc: 47.64%
Epoch 14/50 | loss: 1.4277 | train acc: 48.40% | test acc: 48.94%
Epoch 15/50 | loss: 1.3984 | train acc: 49.59% | test acc: 49.82%
Epoch 16/50 | loss: 1.3708 | train acc: 50.54% | test acc: 49.54%
Epoch 17/50 | loss: 1.3472 | train acc: 51.47% | test acc: 51.65%
Epoch 18/50 | loss: 1.3231 | train acc: 52.32% | test acc: 52.56%
Epoch 19/50 | loss: 1.3005 | train acc: 53.18% | test acc: 52.65%
Epoch 20/50 | loss: 1.2826 | train acc: 53.96% | test acc: 54.19%
Epoch 21/50 | loss: 1.2638 | train acc: 54.75% | test acc: 53.69%
Epoch 22/50 | loss: 1.2468 | train acc: 55.48% | test acc: 54.97%
Epoch 23/50 | loss: 1.2284 | train acc: 56.22% | test acc: 55.44%
Epoch 24/50 | loss: 1.2150 | train acc: 56.80% | test acc: 56.07%
Epoch 25/50 | loss: 1.1978 | train acc: 57.50% | test acc: 55.34%
Epoch 26/50 | loss: 1.1879 | train acc: 57.74% | test acc: 55.79%
Epoch 27/50 | loss: 1.1715 | train acc: 58.47% | test acc: 56.17%
Epoch 28/50 | loss: 1.1570 | train acc: 58.85% | test acc: 57.72%
Epoch 29/50 | loss: 1.1456 | train acc: 59.45% | test acc: 57.59%
Epoch 30/50 | loss: 1.1287 | train acc: 60.09% | test acc: 55.88%
Epoch 31/50 | loss: 1.1212 | train acc: 60.17% | test acc: 58.66%
Epoch 32/50 | loss: 1.1060 | train acc: 60.88% | test acc: 59.18%
```

```
Epoch 33/50 | loss: 1.0954 | train acc: 61.21% | test acc: 59.58%
Epoch 34/50 | loss: 1.0840 | train acc: 61.82% | test acc: 59.41%
Epoch 35/50 | loss: 1.0733 | train acc: 62.28% | test acc: 60.22%
Epoch 36/50 | loss: 1.0616 | train acc: 62.70% | test acc: 59.92%
Epoch 37/50 | loss: 1.0497 | train acc: 63.04% | test acc: 60.86%
Epoch 38/50 | loss: 1.0388 | train acc: 63.50% | test acc: 58.43%
Epoch 39/50 | loss: 1.0298 | train acc: 63.68% | test acc: 61.13%
Epoch 40/50 | loss: 1.0157 | train acc: 64.35% | test acc: 60.95%
Epoch 41/50 | loss: 1.0034 | train acc: 64.83% | test acc: 61.54%
Epoch 42/50 | loss: 0.9937 | train acc: 65.09% | test acc: 61.47%
Epoch 43/50 | loss: 0.9893 | train acc: 65.29% | test acc: 61.95%
Epoch 44/50 | loss: 0.9797 | train acc: 65.68% | test acc: 62.47%
Epoch 45/50 | loss: 0.9672 | train acc: 66.14% | test acc: 61.93%
Epoch 46/50 | loss: 0.9570 | train acc: 66.44% | test acc: 61.91%
Epoch 47/50 | loss: 0.9530 | train acc: 66.67% | test acc: 62.96%
Epoch 48/50 | loss: 0.9423 | train acc: 67.16% | test acc: 62.16%
Epoch 49/50 | loss: 0.9340 | train acc: 67.25% | test acc: 63.02%
Epoch 50/50 | loss: 0.9250 | train acc: 67.67% | test acc: 61.94%
Finished Training
```

```
def run_cnn(net, optimizer, num_epochs: int = 50) -> tuple[list, list, list]:

  from tqdm import tqdm

  train_losses, train_accs, test_accs = [], [], []

  for epoch in range(num_epochs):
      net.train()
      running_loss, correct, total = 0.0, 0, 0

      for inputs, labels in tqdm(trainloader, desc=f'Epoch {epoch + 1}/{num_epochs}', leave=False):
          inputs, labels = inputs.to(device), labels.to(device)

          optimizer.zero_grad()
          outputs = net(inputs)
          loss = criterion(outputs, labels)
          loss.backward()
          optimizer.step()

          running_loss += loss.item()
          correct += (outputs.argmax(dim=1) == labels).sum().item()
          total += labels.size(0)

      train_loss = running_loss / len(trainloader)
      train_acc = 100 * correct / total
      test_acc = evaluate(net, testloader)

      train_losses.append(train_loss)
      train_accs.append(train_acc)
      test_accs.append(test_acc)

      print(f'Epoch {epoch + 1}/{num_epochs} | loss: {train_loss:.4f} | '
            f'train acc: {train_acc:.2f}% | test acc: {test_acc:.2f}%')

  print('Finished Training')

  return train_losses, train_accs, test_accs
```

```
import matplotlib.pyplot as plt

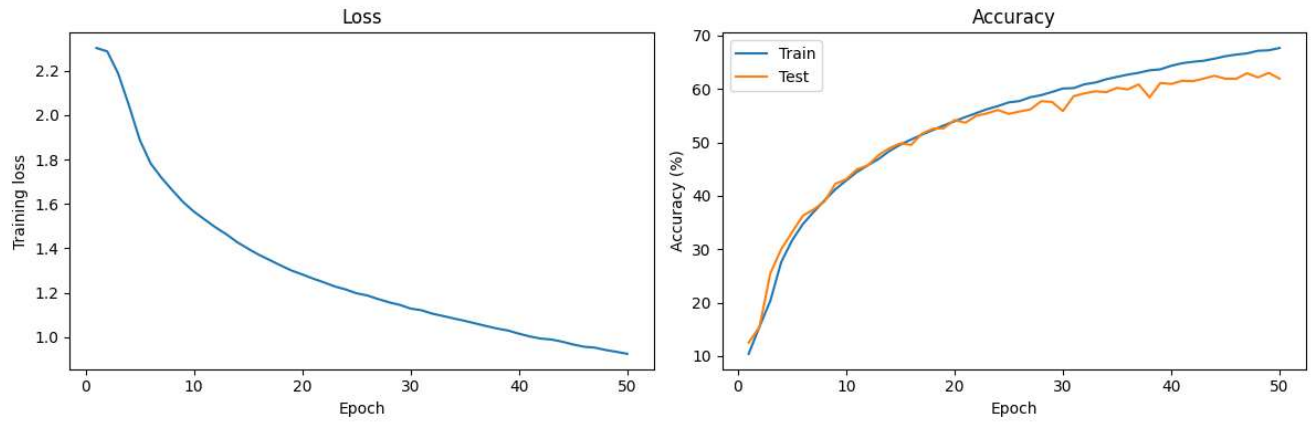
epochs = range(1, num_epochs + 1)

plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(epochs, train_losses)
plt.xlabel('Epoch')
plt.ylabel('Training loss')
plt.title('Loss')

plt.subplot(1, 2, 2)
plt.plot(epochs, train_accs, label='Train')
plt.plot(epochs, test_accs, label='Test')
plt.xlabel('Epoch')
plt.ylabel('Accuracy (%)')
plt.title('Accuracy')
plt.legend()
```

```
plt.tight_layout()
plt.show()
```




```
import matplotlib.pyplot as plt

def plot_results(train_losses: list, train_accs: list, test_accs: list, num_epochs: int=50) -> None:

  epochs = range(1, num_epochs + 1)

  plt.figure(figsize=(12, 4))

  plt.subplot(1, 2, 1)
  plt.plot(epochs, train_losses)
  plt.xlabel('Epoch')
  plt.ylabel('Training loss')
  plt.title('Loss')

  plt.subplot(1, 2, 2)
  plt.plot(epochs, train_accs, label='Train')
  plt.plot(epochs, test_accs, label='Test')
  plt.xlabel('Epoch')
  plt.ylabel('Accuracy (%)')
  plt.title('Accuracy')
  plt.legend()

  plt.tight_layout()
  plt.show()
```

```
dataiter = iter(testloader)
images, labels = next(dataiter)

# print images
imshow(torchvision.utils.make_grid(images[:4]))
plt.axis('off')
plt.show()
print('GroundTruth: ', ' '.join('%5s' % classes[labels[j]] for j in range(4)))

outputs = net(images.to(device))
_, predicted = torch.max(outputs, 1)
print('Predicted: ', ' '.join('%5s' % classes[predicted[j]] for j in range(4)))
```



```
GroundTruth:    cat  ship  ship plane
Predicted:  plane plane plane plane
```

```
correct = 0
total = 0
with torch.no_grad():
    for data in testloader:
        images, labels = data
        labels = labels.to(device)
        outputs = net(images.to(device))
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print('Accuracy of the network on the 10000 test images: %d %%' % (
    100 * correct / total))
```

```
Accuracy of the network on the 10000 test images: 10 %
```

```
class_correct = list(0. for i in range(10))
class_total = list(0. for i in range(10))
with torch.no_grad():
    for data in testloader:
        images, labels = data
        labels = labels.to(device)
        outputs = net(images.to(device))
        _, predicted = torch.max(outputs, 1)
        c = (predicted == labels).squeeze()
        for i in range(4):
            label = labels[i]
            class_correct[label] += c[i].item()
            class_total[label] += 1


for i in range(10):
    print('Accuracy of %5s : %2d %%' % (
        classes[i], 100 * class_correct[i] / class_total[i]))
```

```
Accuracy of plane : 75 %
Accuracy of   car : 82 %
Accuracy of  bird : 60 %
Accuracy of   cat : 41 %
Accuracy of  deer : 40 %
Accuracy of   dog : 51 %
Accuracy of  frog : 83 %
Accuracy of horse : 76 %
Accuracy of  ship : 59 %
Accuracy of truck : 61 %
```

Problem 2

Change the code to have only a single fully connected layer. The model will have a single layer that connects the input to the output. What is the number of parameters? In PyTorch, "nn.Linear" can be used for fully connected layer.

```
class Net_2(nn.Module):
    def __init__(self):
        super(Net_2, self).__init__()
        self.fc = nn.Linear(3 * 32 * 32, 10)

    def forward(self, x):
        x = x.view(-1, 3 * 32 * 32)  # flatten each image
        x = self.fc(x)
        return x

net_2 = Net_2()
```

```
# Put model on GPU
net_2 = Net_2().to(device)
```

```
print(sum(p.numel() for p in net_2.parameters()))
```

```
30730
```

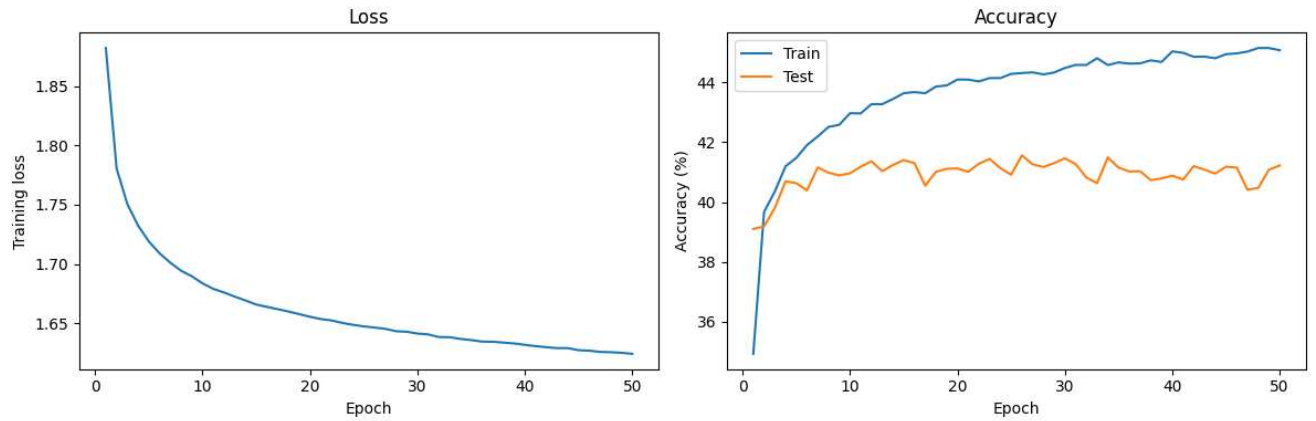
Here we see that there are 3 * 32 * 32 = 30,720 plus 10 biases in the final layer because of the 10 outputs, totaling 30,730.

```
optimizer_2 = optim.SGD(net_2.parameters(), lr=0.001, momentum=0.9)  # use the same settings as the original optimizer

train_losses_2, train_accs_2, test_accs_2 = run_cnn(net=net_2, optimizer=optimizer_2, num_epochs=50)
```

```
Epoch 1/50 | loss: 1.8822 | train acc: 34.93% | test acc: 39.10%
Epoch 2/50 | loss: 1.7806 | train acc: 39.67% | test acc: 39.19%
Epoch 3/50 | loss: 1.7505 | train acc: 40.35% | test acc: 39.80%
Epoch 4/50 | loss: 1.7324 | train acc: 41.19% | test acc: 40.69%
Epoch 5/50 | loss: 1.7190 | train acc: 41.47% | test acc: 40.63%
Epoch 6/50 | loss: 1.7089 | train acc: 41.90% | test acc: 40.39%
Epoch 7/50 | loss: 1.7010 | train acc: 42.19% | test acc: 41.16%
Epoch 8/50 | loss: 1.6943 | train acc: 42.51% | test acc: 40.98%
Epoch 9/50 | loss: 1.6896 | train acc: 42.58% | test acc: 40.89%
Epoch 10/50 | loss: 1.6836 | train acc: 42.97% | test acc: 40.96%
Epoch 11/50 | loss: 1.6790 | train acc: 42.96% | test acc: 41.18%
Epoch 12/50 | loss: 1.6759 | train acc: 43.27% | test acc: 41.36%
Epoch 13/50 | loss: 1.6724 | train acc: 43.27% | test acc: 41.03%
Epoch 14/50 | loss: 1.6692 | train acc: 43.44% | test acc: 41.24%
Epoch 15/50 | loss: 1.6657 | train acc: 43.64% | test acc: 41.40%
Epoch 16/50 | loss: 1.6638 | train acc: 43.67% | test acc: 41.30%
Epoch 17/50 | loss: 1.6618 | train acc: 43.63% | test acc: 40.55%
Epoch 18/50 | loss: 1.6599 | train acc: 43.86% | test acc: 41.01%
Epoch 19/50 | loss: 1.6577 | train acc: 43.90% | test acc: 41.11%
Epoch 20/50 | loss: 1.6554 | train acc: 44.09% | test acc: 41.12%
Epoch 21/50 | loss: 1.6535 | train acc: 44.09% | test acc: 41.01%
Epoch 22/50 | loss: 1.6523 | train acc: 44.03% | test acc: 41.28%
Epoch 23/50 | loss: 1.6503 | train acc: 44.14% | test acc: 41.44%
Epoch 24/50 | loss: 1.6486 | train acc: 44.14% | test acc: 41.13%
Epoch 25/50 | loss: 1.6473 | train acc: 44.28% | test acc: 40.91%
Epoch 26/50 | loss: 1.6463 | train acc: 44.31% | test acc: 41.56%
Epoch 27/50 | loss: 1.6452 | train acc: 44.33% | test acc: 41.26%
Epoch 28/50 | loss: 1.6433 | train acc: 44.26% | test acc: 41.17%
Epoch 29/50 | loss: 1.6428 | train acc: 44.33% | test acc: 41.30%
Epoch 30/50 | loss: 1.6413 | train acc: 44.48% | test acc: 41.46%
Epoch 31/50 | loss: 1.6405 | train acc: 44.58% | test acc: 41.27%
Epoch 32/50 | loss: 1.6383 | train acc: 44.58% | test acc: 40.82%
Epoch 33/50 | loss: 1.6382 | train acc: 44.80% | test acc: 40.63%
Epoch 34/50 | loss: 1.6367 | train acc: 44.58% | test acc: 41.49%
Epoch 35/50 | loss: 1.6358 | train acc: 44.66% | test acc: 41.15%
Epoch 36/50 | loss: 1.6345 | train acc: 44.62% | test acc: 41.02%
Epoch 37/50 | loss: 1.6344 | train acc: 44.63% | test acc: 41.03%
Epoch 38/50 | loss: 1.6337 | train acc: 44.73% | test acc: 40.73%
Epoch 39/50 | loss: 1.6329 | train acc: 44.68% | test acc: 40.79%
Epoch 40/50 | loss: 1.6318 | train acc: 45.03% | test acc: 40.88%
Epoch 41/50 | loss: 1.6307 | train acc: 44.98% | test acc: 40.75%
Epoch 42/50 | loss: 1.6297 | train acc: 44.85% | test acc: 41.20%
Epoch 43/50 | loss: 1.6290 | train acc: 44.86% | test acc: 41.08%
Epoch 44/50 | loss: 1.6290 | train acc: 44.80% | test acc: 40.95%
Epoch 45/50 | loss: 1.6272 | train acc: 44.94% | test acc: 41.18%
Epoch 46/50 | loss: 1.6268 | train acc: 44.96% | test acc: 41.15%
Epoch 47/50 | loss: 1.6258 | train acc: 45.03% | test acc: 40.41%
Epoch 48/50 | loss: 1.6255 | train acc: 45.14% | test acc: 40.47%
Epoch 49/50 | loss: 1.6250 | train acc: 45.14% | test acc: 41.08%
Epoch 50/50 | loss: 1.6242 | train acc: 45.07% | test acc: 41.22%
Finished Training
```

```
plot_results(train_losses_2, train_accs_2, test_accs_2, num_epochs=50)
```



Problem 3

Change the code to have multiple fully connected layers. Try having a layer from input to 110 neurons and then a layer to 74 neurons, and finally a layer to 10 neurons, one for each category. What happens if you do not use ReLU? Describe why.

```
class Net_3(nn.Module):
    def __init__(self):
        super(Net_3, self).__init__()
        self.fc1 = nn.Linear(3 * 32 * 32, 110)
        self.fc2 = nn.Linear(110, 74)
        self.fc3 = nn.Linear(74, 10)

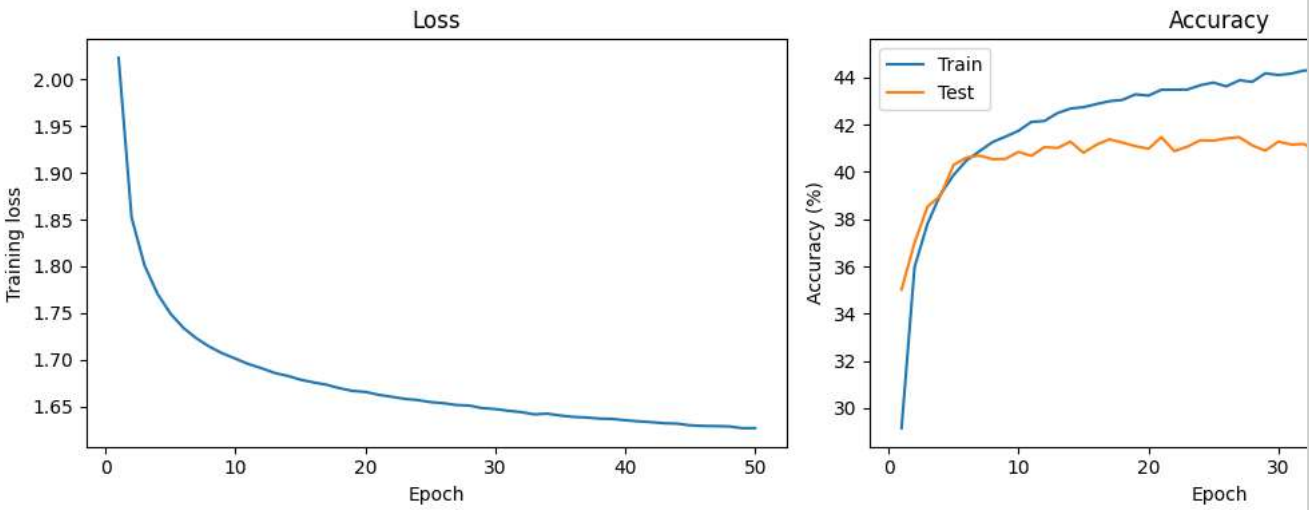
    def forward(self, x):
        x = x.view(-1, 3 * 32 * 32)  # flatten each image
        x = self.fc1(x)
        x = self.fc2(x)
        x = self.fc3(x)              # no ReLU on the output (raw class scores)
        return x
```

```
net_3 = Net_3().to(device)
optimizer_3 = optim.SGD(net_3.parameters(), lr=0.001, momentum=0.9)
```

```
train_losses_3, train_accs_3, test_accs_3 = run_cnn(net=net_3, optimizer=optimizer_3, num_epochs=50)
```

```
Epoch 1/50 | loss: 2.0233 | train acc: 29.15% | test acc: 35.02%
Epoch 2/50 | loss: 1.8523 | train acc: 35.97% | test acc: 37.00%
Epoch 3/50 | loss: 1.8010 | train acc: 37.81% | test acc: 38.53%
Epoch 4/50 | loss: 1.7705 | train acc: 39.06% | test acc: 39.00%
Epoch 5/50 | loss: 1.7493 | train acc: 39.86% | test acc: 40.29%
Epoch 6/50 | loss: 1.7339 | train acc: 40.48% | test acc: 40.61%
Epoch 7/50 | loss: 1.7228 | train acc: 40.88% | test acc: 40.69%
Epoch 8/50 | loss: 1.7139 | train acc: 41.26% | test acc: 40.54%
Epoch 9/50 | loss: 1.7067 | train acc: 41.49% | test acc: 40.55%
Epoch 10/50 | loss: 1.7011 | train acc: 41.74% | test acc: 40.85%
Epoch 11/50 | loss: 1.6953 | train acc: 42.12% | test acc: 40.68%
Epoch 12/50 | loss: 1.6909 | train acc: 42.16% | test acc: 41.05%
Epoch 13/50 | loss: 1.6859 | train acc: 42.49% | test acc: 41.01%
Epoch 14/50 | loss: 1.6827 | train acc: 42.68% | test acc: 41.28%
Epoch 15/50 | loss: 1.6786 | train acc: 42.74% | test acc: 40.81%
Epoch 16/50 | loss: 1.6757 | train acc: 42.87% | test acc: 41.15%
Epoch 17/50 | loss: 1.6732 | train acc: 42.99% | test acc: 41.38%
Epoch 18/50 | loss: 1.6695 | train acc: 43.05% | test acc: 41.24%
Epoch 19/50 | loss: 1.6666 | train acc: 43.28% | test acc: 41.09%
Epoch 20/50 | loss: 1.6655 | train acc: 43.23% | test acc: 40.98%
Epoch 21/50 | loss: 1.6625 | train acc: 43.48% | test acc: 41.47%
Epoch 22/50 | loss: 1.6603 | train acc: 43.48% | test acc: 40.88%
Epoch 23/50 | loss: 1.6581 | train acc: 43.49% | test acc: 41.07%
Epoch 24/50 | loss: 1.6568 | train acc: 43.67% | test acc: 41.34%
Epoch 25/50 | loss: 1.6546 | train acc: 43.78% | test acc: 41.32%
Epoch 26/50 | loss: 1.6535 | train acc: 43.62% | test acc: 41.42%
Epoch 27/50 | loss: 1.6515 | train acc: 43.88% | test acc: 41.47%
Epoch 28/50 | loss: 1.6509 | train acc: 43.81% | test acc: 41.12%
Epoch 29/50 | loss: 1.6481 | train acc: 44.17% | test acc: 40.90%
Epoch 30/50 | loss: 1.6472 | train acc: 44.10% | test acc: 41.28%
Epoch 31/50 | loss: 1.6453 | train acc: 44.16% | test acc: 41.16%
Epoch 32/50 | loss: 1.6438 | train acc: 44.29% | test acc: 41.18%
Epoch 33/50 | loss: 1.6415 | train acc: 44.30% | test acc: 40.87%
Epoch 34/50 | loss: 1.6422 | train acc: 44.27% | test acc: 40.80%
Epoch 35/50 | loss: 1.6402 | train acc: 44.46% | test acc: 41.13%
Epoch 36/50 | loss: 1.6387 | train acc: 44.53% | test acc: 40.88%
Epoch 37/50 | loss: 1.6381 | train acc: 44.43% | test acc: 40.71%
Epoch 38/50 | loss: 1.6369 | train acc: 44.39% | test acc: 40.92%
Epoch 39/50 | loss: 1.6366 | train acc: 44.41% | test acc: 40.42%
Epoch 40/50 | loss: 1.6352 | train acc: 44.55% | test acc: 41.05%
Epoch 41/50 | loss: 1.6341 | train acc: 44.58% | test acc: 40.38%
Epoch 42/50 | loss: 1.6331 | train acc: 44.59% | test acc: 40.98%
Epoch 43/50 | loss: 1.6320 | train acc: 44.65% | test acc: 41.29%
Epoch 44/50 | loss: 1.6315 | train acc: 44.77% | test acc: 40.80%
Epoch 45/50 | loss: 1.6297 | train acc: 44.82% | test acc: 40.96%
Epoch 46/50 | loss: 1.6291 | train acc: 44.77% | test acc: 40.50%
Epoch 47/50 | loss: 1.6289 | train acc: 44.75% | test acc: 40.76%
Epoch 48/50 | loss: 1.6286 | train acc: 44.74% | test acc: 41.06%
Epoch 49/50 | loss: 1.6267 | train acc: 44.82% | test acc: 40.53%
Epoch 50/50 | loss: 1.6267 | train acc: 44.83% | test acc: 40.35%
Finished Training
```

```
plot_results(train_losses_3, train_accs_3, test_accs_3, num_epochs=50)
```



Without ReLU activation function, any multi-layer neural network is just a linear combination of each layer which can be represented by a vector. This is why the results are comparable to the 2 layer neural network in problem 2.

Problem 4

Change the code by adding two convolutional layers along with maxpooling layers before the fully connected layers. This will be similar to the example in the tutorial. Use this model for the following sections.

```
class Net_4(nn.Module):
    def __init__(self):
        super(Net_4, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 110)
        self.fc2 = nn.Linear(110, 74)
        self.fc3 = nn.Linear(74, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 16 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

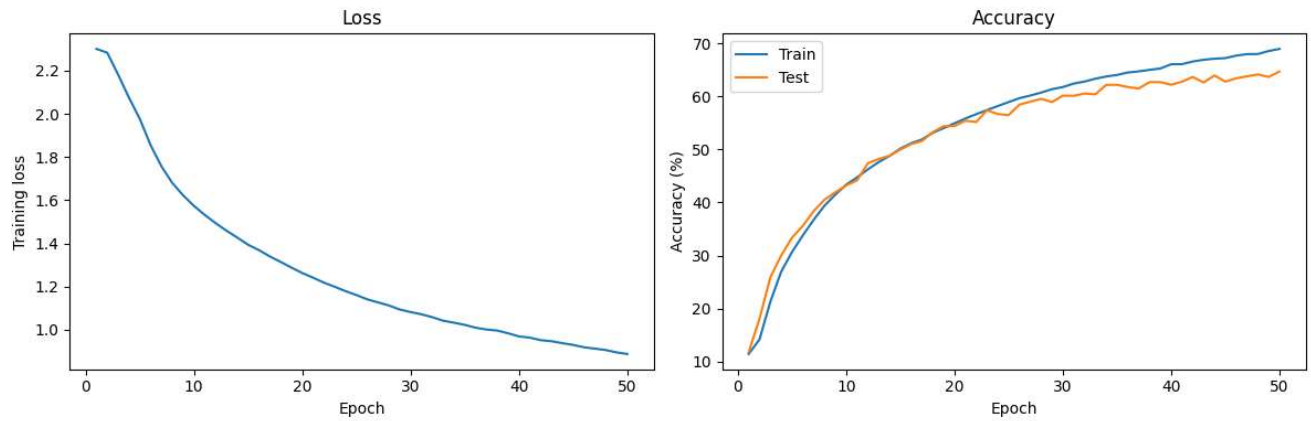
```
net_4 = Net_4().to(device)
optimizer_4 = optim.SGD(net_4.parameters(), lr=0.001, momentum=0.9)  # match the earlier settings
```

```
train_losses_4, train_accs_4, test_accs_4 = run_cnn(net=net_4, optimizer=optimizer_4, num_epochs=50)
```

```
Epoch 1/50 | loss: 2.3009 | train acc: 11.44% | test acc: 11.77%
Epoch 2/50 | loss: 2.2835 | train acc: 14.20% | test acc: 18.10%
Epoch 3/50 | loss: 2.1826 | train acc: 21.31% | test acc: 25.89%
Epoch 4/50 | loss: 2.0759 | train acc: 26.96% | test acc: 30.00%
Epoch 5/50 | loss: 1.9776 | train acc: 30.67% | test acc: 33.37%
Epoch 6/50 | loss: 1.8557 | train acc: 33.81% | test acc: 35.61%
Epoch 7/50 | loss: 1.7575 | train acc: 36.74% | test acc: 38.37%
Epoch 8/50 | loss: 1.6805 | train acc: 39.41% | test acc: 40.54%
Epoch 9/50 | loss: 1.6231 | train acc: 41.49% | test acc: 41.92%
Epoch 10/50 | loss: 1.5741 | train acc: 43.36% | test acc: 43.22%
Epoch 11/50 | loss: 1.5320 | train acc: 44.73% | test acc: 44.15%
Epoch 12/50 | loss: 1.4943 | train acc: 46.24% | test acc: 47.40%
Epoch 13/50 | loss: 1.4592 | train acc: 47.61% | test acc: 48.19%
Epoch 14/50 | loss: 1.4274 | train acc: 48.78% | test acc: 48.81%
Epoch 15/50 | loss: 1.3939 | train acc: 50.15% | test acc: 49.97%
Epoch 16/50 | loss: 1.3691 | train acc: 51.18% | test acc: 51.00%
Epoch 17/50 | loss: 1.3402 | train acc: 51.89% | test acc: 51.60%
Epoch 18/50 | loss: 1.3146 | train acc: 53.14% | test acc: 53.27%
Epoch 19/50 | loss: 1.2883 | train acc: 54.02% | test acc: 54.38%
Epoch 20/50 | loss: 1.2629 | train acc: 54.92% | test acc: 54.40%
```

```
Epoch 21/50 | loss: 1.2415 | train acc: 55.83% | test acc: 55.40%
Epoch 22/50 | loss: 1.2184 | train acc: 56.64% | test acc: 55.17%
Epoch 23/50 | loss: 1.1992 | train acc: 57.42% | test acc: 57.37%
Epoch 24/50 | loss: 1.1790 | train acc: 58.18% | test acc: 56.67%
Epoch 25/50 | loss: 1.1609 | train acc: 58.93% | test acc: 56.47%
Epoch 26/50 | loss: 1.1411 | train acc: 59.66% | test acc: 58.45%
Epoch 27/50 | loss: 1.1269 | train acc: 60.16% | test acc: 59.00%
Epoch 28/50 | loss: 1.1127 | train acc: 60.71% | test acc: 59.53%
Epoch 29/50 | loss: 1.0939 | train acc: 61.36% | test acc: 58.94%
Epoch 30/50 | loss: 1.0823 | train acc: 61.76% | test acc: 60.13%
Epoch 31/50 | loss: 1.0719 | train acc: 62.41% | test acc: 60.11%
Epoch 32/50 | loss: 1.0586 | train acc: 62.80% | test acc: 60.53%
Epoch 33/50 | loss: 1.0422 | train acc: 63.33% | test acc: 60.38%
Epoch 34/50 | loss: 1.0330 | train acc: 63.75% | test acc: 62.16%
Epoch 35/50 | loss: 1.0228 | train acc: 64.04% | test acc: 62.18%
Epoch 36/50 | loss: 1.0098 | train acc: 64.50% | test acc: 61.76%
Epoch 37/50 | loss: 1.0013 | train acc: 64.71% | test acc: 61.48%
Epoch 38/50 | loss: 0.9965 | train acc: 64.99% | test acc: 62.68%
Epoch 39/50 | loss: 0.9841 | train acc: 65.26% | test acc: 62.69%
Epoch 40/50 | loss: 0.9695 | train acc: 66.05% | test acc: 62.19%
Epoch 41/50 | loss: 0.9639 | train acc: 66.07% | test acc: 62.77%
Epoch 42/50 | loss: 0.9521 | train acc: 66.57% | test acc: 63.66%
Epoch 43/50 | loss: 0.9471 | train acc: 66.90% | test acc: 62.62%
Epoch 44/50 | loss: 0.9381 | train acc: 67.10% | test acc: 63.94%
Epoch 45/50 | loss: 0.9301 | train acc: 67.21% | test acc: 62.78%
Epoch 46/50 | loss: 0.9191 | train acc: 67.68% | test acc: 63.40%
Epoch 47/50 | loss: 0.9129 | train acc: 67.96% | test acc: 63.78%
Epoch 48/50 | loss: 0.9065 | train acc: 67.99% | test acc: 64.13%
Epoch 49/50 | loss: 0.8955 | train acc: 68.55% | test acc: 63.66%
Epoch 50/50 | loss: 0.8879 | train acc: 68.95% | test acc: 64.68%
Finished Training
```

```
plot_results(train_losses_4, train_accs_4, test_accs_4, num_epochs=50)
```



Here we see the train and test accuracy both increase back up to levels seen in problem 1. The max pooling and convolution layers prove their clear need and easily out preform single layer neural networks and its equivalent multilayered neural network with no activation function.

The slight difference in test and training accuracies between this CNN and the CNN in problem 1 is due to different layer widths and initialization randomness.

Problem 5

Try multiple batch sizes to see the effect and describe the findings. Please use batch size of 1, 4, and 1000. If 1000 does not fit into the memory of your machine, please feel free to reduce it to the largest possible number.

```
import time
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
from tqdm import tqdm
```

```
def evaluate(model, loader):
    model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            correct += (model(images).argmax(dim=1) == labels).sum().item()
            total += labels.size(0)
    return 100 * correct / total
```

```
def run_experiment(batch_size, num_epochs, lr=0.001, momentum=0.9):
    trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,
                                              shuffle=True, num_workers=2, pin_memory=True)
    testloader = torch.utils.data.DataLoader(testset, batch_size=1000,
                                             shuffle=False, num_workers=2, pin_memory=True)

    net = Net_4().to(device)                      # fresh model for every run
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD(net.parameters(), lr=lr, momentum=momentum)

    hist = {'loss': [], 'train_acc': [], 'test_acc': [], 'time': []}
    start = time.time()

    for epoch in range(num_epochs):
        net.train()
        running_loss, correct, total = 0.0, 0, 0

        for inputs, labels in tqdm(trainloader, desc=f'bs={batch_size} epoch {epoch + 1}/{num_epochs}', leave=False):
            inputs, labels = inputs.to(device), labels.to(device)

            optimizer.zero_grad()
            outputs = net(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item()
            correct += (outputs.argmax(dim=1) == labels).sum().item()
            total += labels.size(0)

        hist['loss'].append(running_loss / len(trainloader))
        hist['train_acc'].append(100 * correct / total)
        hist['test_acc'].append(evaluate(net, testloader))
        hist['time'].append(time.time() - start)

        print(f'bs={batch_size} | epoch {epoch + 1}/{num_epochs} | loss: {hist["loss"][-1]:.4f} | '
              f'train acc: {hist["train_acc"][-1]:.2f}% | test acc: {hist["test_acc"][-1]:.2f}% | '
              f'elapsed: {hist["time"][-1]:.0f}s')

    return hist
```

```
results = {}
for bs in [1000, 4, 1]:
    results[bs] = run_experiment(bs, num_epochs=10)
```

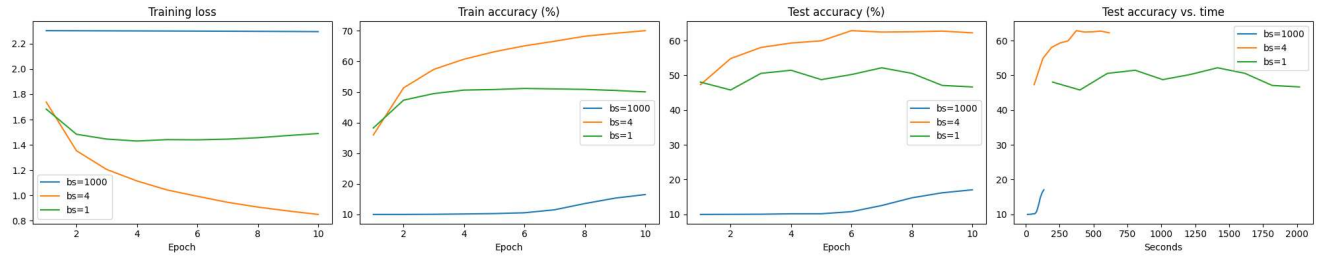
```
bs=1000 | epoch 1/10 | loss: 2.3037 | train acc: 10.00% | test acc: 9.99% | elapsed: 14s
bs=1000 | epoch 2/10 | loss: 2.3030 | train acc: 10.00% | test acc: 10.02% | elapsed: 27s
bs=1000 | epoch 3/10 | loss: 2.3023 | train acc: 10.05% | test acc: 10.05% | elapsed: 40s
bs=1000 | epoch 4/10 | loss: 2.3016 | train acc: 10.15% | test acc: 10.17% | elapsed: 53s
bs=1000 | epoch 5/10 | loss: 2.3009 | train acc: 10.28% | test acc: 10.18% | elapsed: 67s
bs=1000 | epoch 6/10 | loss: 2.3001 | train acc: 10.54% | test acc: 10.79% | elapsed: 80s
bs=1000 | epoch 7/10 | loss: 2.2993 | train acc: 11.52% | test acc: 12.55% | elapsed: 94s
bs=1000 | epoch 8/10 | loss: 2.2983 | train acc: 13.56% | test acc: 14.77% | elapsed: 107s
bs=1000 | epoch 9/10 | loss: 2.2972 | train acc: 15.32% | test acc: 16.21% | elapsed: 120s
bs=1000 | epoch 10/10 | loss: 2.2959 | train acc: 16.49% | test acc: 17.07% | elapsed: 134s
bs=4 | epoch 1/10 | loss: 1.7383 | train acc: 35.96% | test acc: 47.31% | elapsed: 62s
bs=4 | epoch 2/10 | loss: 1.3533 | train acc: 51.35% | test acc: 54.81% | elapsed: 126s
bs=4 | epoch 3/10 | loss: 1.2052 | train acc: 57.39% | test acc: 58.01% | elapsed: 191s
bs=4 | epoch 4/10 | loss: 1.1145 | train acc: 60.72% | test acc: 59.28% | elapsed: 252s
bs=4 | epoch 5/10 | loss: 1.0430 | train acc: 63.14% | test acc: 59.92% | elapsed: 313s
bs=4 | epoch 6/10 | loss: 0.9929 | train acc: 65.07% | test acc: 62.84% | elapsed: 373s
bs=4 | epoch 7/10 | loss: 0.9453 | train acc: 66.59% | test acc: 62.44% | elapsed: 434s
bs=4 | epoch 8/10 | loss: 0.9076 | train acc: 68.22% | test acc: 62.52% | elapsed: 494s
bs=4 | epoch 9/10 | loss: 0.8768 | train acc: 69.18% | test acc: 62.71% | elapsed: 555s
bs=4 | epoch 10/10 | loss: 0.8493 | train acc: 70.04% | test acc: 62.22% | elapsed: 616s
bs=1 | epoch 1/10 | loss: 1.6830 | train acc: 38.26% | test acc: 48.04% | elapsed: 199s
bs=1 | epoch 2/10 | loss: 1.4837 | train acc: 47.33% | test acc: 45.78% | elapsed: 400s
bs=1 | epoch 3/10 | loss: 1.4456 | train acc: 49.48% | test acc: 50.55% | elapsed: 602s
```

```
bs=1 | epoch 4/10 | loss: 1.4302 | train acc: 50.62% | test acc: 51.45% | elapsed: 807s
bs=1 | epoch 5/10 | loss: 1.4412 | train acc: 50.82% | test acc: 48.76% | elapsed: 1011s
bs=1 | epoch 6/10 | loss: 1.4396 | train acc: 51.15% | test acc: 50.22% | elapsed: 1212s
bs=1 | epoch 7/10 | loss: 1.4451 | train acc: 51.00% | test acc: 52.16% | elapsed: 1415s
bs=1 | epoch 8/10 | loss: 1.4562 | train acc: 50.86% | test acc: 50.56% | elapsed: 1617s
bs=1 | epoch 9/10 | loss: 1.4736 | train acc: 50.50% | test acc: 47.10% | elapsed: 1817s
bs=1 | epoch 10/10 | loss: 1.4896 | train acc: 50.05% | test acc: 46.64% | elapsed: 2019s
```

```
fig, ax = plt.subplots(1, 4, figsize=(20, 4))
for bs, h in results.items():
    ep = range(1, len(h['loss']) + 1)
    ax[0].plot(ep, h['loss'], label=f'bs={bs}')
    ax[1].plot(ep, h['train_acc'], label=f'bs={bs}')
    ax[2].plot(ep, h['test_acc'], label=f'bs={bs}')
    ax[3].plot(h['time'], h['test_acc'], label=f'bs={bs}')

for a, t, x in zip(ax, ['Training loss', 'Train accuracy (%)', 'Test accuracy (%)', 'Test accuracy vs. time'],
                   ['Epoch', 'Epoch', 'Epoch', 'Seconds']):
    a.set_title(t); a.set_xlabel(x); a.legend()
plt.tight_layout()
plt.show()

for bs, h in results.items():
    print(f'bs={bs:>4} | final test acc: {h["test_acc"][-1]:.2f}% | total time: {h["time"][-1]:.0f}s | '
          f'avg time/epoch: {h["time"][-1] / len(h["time"]):.0f}s')
```




```
bs=1000 | final test acc: 17.07% | total time: 134s | avg time/epoch: 13s
bs=   4 | final test acc: 62.22% | total time: 616s | avg time/epoch: 62s
bs=   1 | final test acc: 46.64% | total time: 2019s | avg time/epoch: 202s
```

The training loss for batch size=1000 does not change much, potentially because with such a large batch size, each iteration through the forward and backward pass does not update the weights much. We see for batch size=1 it has minor improvements however for a reasonable choice like 4, it decreases after 10 epochs.

The train accuracy plot is similar to a vertically flipped training loss plot. I notice for the large batch size, accuracy does not increase much because each iteration yields very small update weights. Batch size = 1 shows some improvement on weights after each epoch but plateaus, while 4 shows a clear upward trend.

Test accuracy follows a similar pattern with train accuracy, with some slight variations. Once again batch size = 4 performs the best.

It is worth noting that batch size = 1 has the longest run time because with 1 training point, it is forcing a forward and backward pass once for each training point, thus 50,000 steps. Far larger than if we chose 4 or 1000.

Problem 6

Try multiple learning rates to see the effect and describe the findings. Please use learning rates of 10, 0.1, 0.01, and 0.001.

```
BATCH_SIZE = 128
NUM_EPOCHS = 50

lr_results = {}
for lr in [10, 0.1, 0.01, 0.001]:
```

```
    print(f'\n===== lr = {lr} =====')
    lr_results[lr] = run_experiment(BATCH_SIZE, num_epochs=NUM_EPOCHS, lr=lr)
```

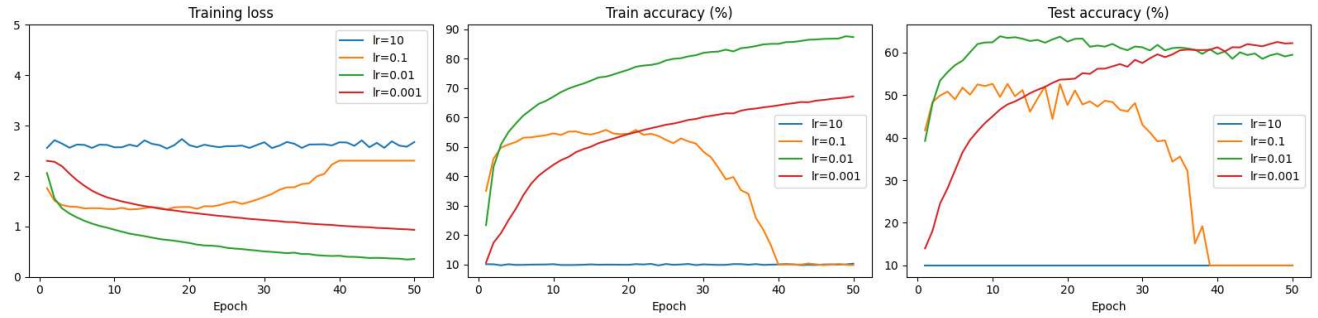
```
bs=128 | epoch 23/50 | loss: 0.6183 | train acc: 77.83% | test acc: 61.35% | elapsed: 651s
bs=128 | epoch 24/50 | loss: 0.6049 | train acc: 78.33% | test acc: 61.63% | elapsed: 678s
bs=128 | epoch 25/50 | loss: 0.5767 | train acc: 79.35% | test acc: 61.39% | elapsed: 706s
bs=128 | epoch 26/50 | loss: 0.5617 | train acc: 79.89% | test acc: 62.02% | elapsed: 735s
bs=128 | epoch 27/50 | loss: 0.5512 | train acc: 80.11% | test acc: 61.08% | elapsed: 764s
bs=128 | epoch 28/50 | loss: 0.5348 | train acc: 80.74% | test acc: 60.54% | elapsed: 793s
bs=128 | epoch 29/50 | loss: 0.5219 | train acc: 81.13% | test acc: 61.41% | elapsed: 819s
bs=128 | epoch 30/50 | loss: 0.5041 | train acc: 81.92% | test acc: 61.22% | elapsed: 849s
bs=128 | epoch 31/50 | loss: 0.4955 | train acc: 82.22% | test acc: 60.48% | elapsed: 877s
bs=128 | epoch 32/50 | loss: 0.4827 | train acc: 82.38% | test acc: 61.80% | elapsed: 906s
bs=128 | epoch 33/50 | loss: 0.4707 | train acc: 83.01% | test acc: 60.50% | elapsed: 935s
bs=128 | epoch 34/50 | loss: 0.4802 | train acc: 82.41% | test acc: 61.03% | elapsed: 961s
bs=128 | epoch 35/50 | loss: 0.4538 | train acc: 83.50% | test acc: 61.17% | elapsed: 990s
bs=128 | epoch 36/50 | loss: 0.4518 | train acc: 83.79% | test acc: 60.95% | elapsed: 1018s
bs=128 | epoch 37/50 | loss: 0.4296 | train acc: 84.24% | test acc: 60.64% | elapsed: 1047s
bs=128 | epoch 38/50 | loss: 0.4205 | train acc: 84.79% | test acc: 59.69% | elapsed: 1075s
bs=128 | epoch 39/50 | loss: 0.4141 | train acc: 84.99% | test acc: 60.75% | elapsed: 1102s
bs=128 | epoch 40/50 | loss: 0.4194 | train acc: 84.98% | test acc: 59.64% | elapsed: 1131s
bs=128 | epoch 41/50 | loss: 0.3983 | train acc: 85.56% | test acc: 60.25% | elapsed: 1159s
bs=128 | epoch 42/50 | loss: 0.3967 | train acc: 85.65% | test acc: 58.53% | elapsed: 1188s
bs=128 | epoch 43/50 | loss: 0.3873 | train acc: 85.95% | test acc: 60.04% | elapsed: 1214s
bs=128 | epoch 44/50 | loss: 0.3746 | train acc: 86.37% | test acc: 59.41% | elapsed: 1243s
bs=128 | epoch 45/50 | loss: 0.3774 | train acc: 86.46% | test acc: 59.76% | elapsed: 1271s
bs=128 | epoch 46/50 | loss: 0.3728 | train acc: 86.67% | test acc: 58.52% | elapsed: 1300s
bs=128 | epoch 47/50 | loss: 0.3640 | train acc: 86.73% | test acc: 59.31% | elapsed: 1329s
bs=128 | epoch 48/50 | loss: 0.3610 | train acc: 86.78% | test acc: 59.73% | elapsed: 1355s
bs=128 | epoch 49/50 | loss: 0.3440 | train acc: 87.57% | test acc: 59.08% | elapsed: 1384s
bs=128 | epoch 50/50 | loss: 0.3548 | train acc: 87.29% | test acc: 59.47% | elapsed: 1412s

===== lr = 0.001 =====
bs=128 | epoch 1/50 | loss: 2.3011 | train acc: 10.63% | test acc: 14.00% | elapsed: 29s
bs=128 | epoch 2/50 | loss: 2.2818 | train acc: 17.41% | test acc: 18.15% | elapsed: 57s
bs=128 | epoch 3/50 | loss: 2.1935 | train acc: 20.69% | test acc: 24.52% | elapsed: 84s
bs=128 | epoch 4/50 | loss: 2.0482 | train acc: 25.08% | test acc: 28.11% | elapsed: 112s
bs=128 | epoch 5/50 | loss: 1.9159 | train acc: 29.03% | test acc: 32.33% | elapsed: 141s
bs=128 | epoch 6/50 | loss: 1.8056 | train acc: 33.67% | test acc: 36.56% | elapsed: 170s
bs=128 | epoch 7/50 | loss: 1.7098 | train acc: 37.39% | test acc: 39.45% | elapsed: 199s
bs=128 | epoch 8/50 | loss: 1.6366 | train acc: 40.13% | test acc: 41.54% | elapsed: 226s
bs=128 | epoch 9/50 | loss: 1.5780 | train acc: 42.12% | test acc: 43.43% | elapsed: 255s
bs=128 | epoch 10/50 | loss: 1.5360 | train acc: 43.90% | test acc: 45.01% | elapsed: 284s
bs=128 | epoch 11/50 | loss: 1.4960 | train acc: 45.45% | test acc: 46.61% | elapsed: 313s
bs=128 | epoch 12/50 | loss: 1.4648 | train acc: 46.54% | test acc: 47.82% | elapsed: 341s
bs=128 | epoch 13/50 | loss: 1.4307 | train acc: 48.18% | test acc: 48.50% | elapsed: 368s
bs=128 | epoch 14/50 | loss: 1.4040 | train acc: 49.21% | test acc: 49.41% | elapsed: 396s
bs=128 | epoch 15/50 | loss: 1.3804 | train acc: 50.04% | test acc: 50.44% | elapsed: 425s
bs=128 | epoch 16/50 | loss: 1.3540 | train acc: 51.23% | test acc: 51.25% | elapsed: 454s
bs=128 | epoch 17/50 | loss: 1.3331 | train acc: 52.03% | test acc: 51.93% | elapsed: 482s
bs=128 | epoch 18/50 | loss: 1.3172 | train acc: 52.77% | test acc: 52.87% | elapsed: 509s
bs=128 | epoch 19/50 | loss: 1.2949 | train acc: 53.56% | test acc: 53.62% | elapsed: 538s
bs=128 | epoch 20/50 | loss: 1.2768 | train acc: 54.32% | test acc: 53.71% | elapsed: 567s
bs=128 | epoch 21/50 | loss: 1.2600 | train acc: 54.97% | test acc: 53.87% | elapsed: 595s
bs=128 | epoch 22/50 | loss: 1.2422 | train acc: 55.71% | test acc: 55.14% | elapsed: 623s
bs=128 | epoch 23/50 | loss: 1.2265 | train acc: 56.31% | test acc: 54.99% | elapsed: 650s
bs=128 | epoch 24/50 | loss: 1.2094 | train acc: 56.88% | test acc: 56.18% | elapsed: 679s
bs=128 | epoch 25/50 | loss: 1.1964 | train acc: 57.49% | test acc: 56.21% | elapsed: 708s
bs=128 | epoch 26/50 | loss: 1.1803 | train acc: 57.90% | test acc: 56.75% | elapsed: 737s
bs=128 | epoch 27/50 | loss: 1.1672 | train acc: 58.46% | test acc: 57.28% | elapsed: 765s
```

```
fig, ax = plt.subplots(1, 3, figsize=(16, 4))
for lr, h in lr_results.items():
    ep = range(1, len(h['loss']) + 1)
    ax[0].plot(ep, h['loss'], label=f'lr={lr}')
    ax[1].plot(ep, h['train_acc'], label=f'lr={lr}')
    ax[2].plot(ep, h['test_acc'], label=f'lr={lr}')

ax[0].set_title('Training loss');       ax[0].set_ylim(0, 5)   # lr=10 can explode, so clip the axis
ax[1].set_title('Train accuracy (%)')
ax[2].set_title('Test accuracy (%)')
for a in ax:
    a.set_xlabel('Epoch'); a.legend()
plt.tight_layout()
plt.show()

for lr, h in lr_results.items():
    print(f'lr={lr:<6} | final loss: {h["loss"][-1]:.4f} | final train acc: {h["train_acc"][-1]:.2f}% | '
          f'final test acc: {h["test_acc"][-1]:.2f}% | best test acc: {max(h["test_acc"]):.2f}%')
```




```
lr=10     | final loss: 2.6736 | final train acc: 10.29% | final test acc: 10.00% | best test acc: 10.00%
lr=0.1    | final loss: 2.3043 | final train acc: 9.79% | final test acc: 10.00% | best test acc: 52.65%
lr=0.01   | final loss: 0.3548 | final train acc: 87.29% | final test acc: 59.47% | best test acc: 63.82%
lr=0.001  | final loss: 0.9321 | final train acc: 67.14% | final test acc: 62.19% | best test acc: 62.47%
```

Here we see staggering performances from larger learning rates on the training loss, with a reasonable step size for gradient descent and back propagation being about 0.01. This holds for training and test accuracy as well, since the lr=0.01 outperforms others quite easily.

Problem 7

Please add some data augmentation to avoid overfitting. Note that you need to do this only for the trainnig and not the testing. You may use line 244-260 from Imagenet sample code:

https://github.com/pytorch/examples/blob/master/imagenet/main.py

```
normalize = transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))

train_transform = transforms.Compose([
    transforms.RandomCrop(32, padding=4),   # CIFAR version of random crop
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    normalize,
])

test_transform = transforms.Compose([
    transforms.ToTensor(),
    normalize,
])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                        download=False, transform=train_transform)
testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=False, transform=test_transform)

trainloader = torch.utils.data.DataLoader(trainset, batch_size=128, shuffle=True, num_workers=2, pin_memory=True)
testloader = torch.utils.data.DataLoader(testset, batch_size=128, shuffle=False, num_workers=2, pin_memory=True)
```

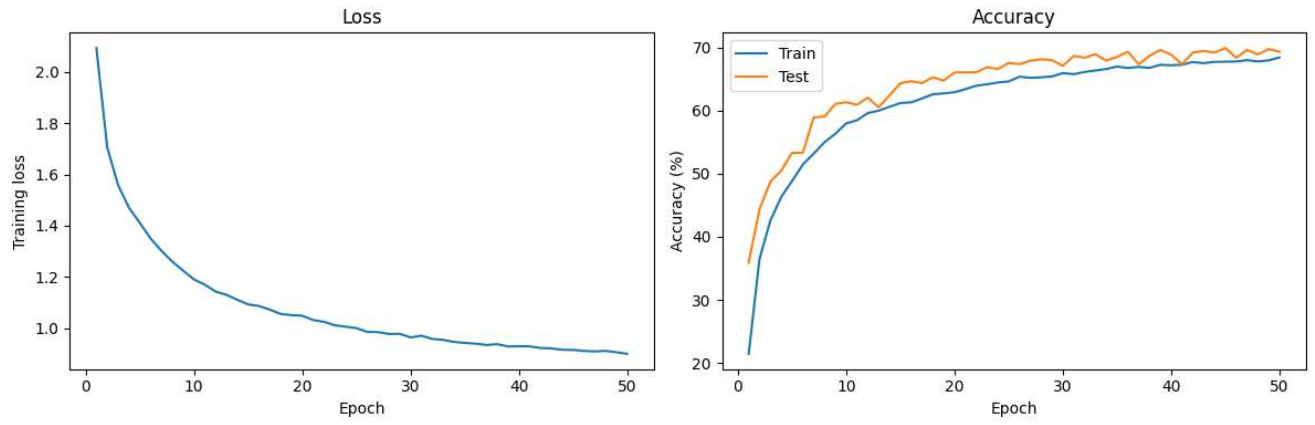
```
net_4 = Net_4().to(device)
optimizer_4 = optim.SGD(net_4.parameters(), lr=0.01, momentum=0.9)
```

```
train_losses_7, train_accs_7, test_accs_7 = run_cnn(net=net_4, optimizer=optimizer_4, num_epochs=50)
```

```
Epoch 1/50:   0%|          | 0/391 [00:00<?, ?it/s]/usr/local/lib/python3.13/dist-packages/torch/utils/data/dataloader.py:1118:
  super().__init__(loader)
Epoch 1/50 | loss: 2.0925 | train acc: 21.46% | test acc: 35.88%
Epoch 2/50 | loss: 1.7053 | train acc: 36.52% | test acc: 44.36%
Epoch 3/50 | loss: 1.5592 | train acc: 42.56% | test acc: 48.72%
Epoch 4/50 | loss: 1.4707 | train acc: 46.31% | test acc: 50.47%
Epoch 5/50 | loss: 1.4117 | train acc: 48.83% | test acc: 53.29%
Epoch 6/50 | loss: 1.3504 | train acc: 51.45% | test acc: 53.32%
Epoch 7/50 | loss: 1.3027 | train acc: 53.17% | test acc: 58.92%
Epoch 8/50 | loss: 1.2606 | train acc: 54.98% | test acc: 59.03%
```

```
Epoch 9/50 | loss: 1.2253 | train acc: 56.32% | test acc: 61.04%
Epoch 10/50 | loss: 1.1908 | train acc: 57.95% | test acc: 61.31%
Epoch 11/50 | loss: 1.1707 | train acc: 58.46% | test acc: 60.91%
Epoch 12/50 | loss: 1.1435 | train acc: 59.60% | test acc: 62.06%
Epoch 13/50 | loss: 1.1311 | train acc: 59.98% | test acc: 60.51%
Epoch 14/50 | loss: 1.1116 | train acc: 60.59% | test acc: 62.38%
Epoch 15/50 | loss: 1.0937 | train acc: 61.19% | test acc: 64.31%
Epoch 16/50 | loss: 1.0874 | train acc: 61.29% | test acc: 64.64%
Epoch 17/50 | loss: 1.0731 | train acc: 61.91% | test acc: 64.34%
Epoch 18/50 | loss: 1.0564 | train acc: 62.58% | test acc: 65.26%
Epoch 19/50 | loss: 1.0518 | train acc: 62.71% | test acc: 64.74%
Epoch 20/50 | loss: 1.0494 | train acc: 62.90% | test acc: 66.03%
Epoch 21/50 | loss: 1.0326 | train acc: 63.38% | test acc: 66.04%
Epoch 22/50 | loss: 1.0258 | train acc: 63.93% | test acc: 66.05%
Epoch 23/50 | loss: 1.0122 | train acc: 64.15% | test acc: 66.87%
Epoch 24/50 | loss: 1.0067 | train acc: 64.44% | test acc: 66.57%
Epoch 25/50 | loss: 1.0010 | train acc: 64.60% | test acc: 67.54%
Epoch 26/50 | loss: 0.9863 | train acc: 65.36% | test acc: 67.35%
Epoch 27/50 | loss: 0.9856 | train acc: 65.17% | test acc: 67.88%
Epoch 28/50 | loss: 0.9783 | train acc: 65.26% | test acc: 68.12%
Epoch 29/50 | loss: 0.9789 | train acc: 65.41% | test acc: 67.97%
Epoch 30/50 | loss: 0.9649 | train acc: 65.92% | test acc: 67.08%
Epoch 31/50 | loss: 0.9718 | train acc: 65.76% | test acc: 68.65%
Epoch 32/50 | loss: 0.9592 | train acc: 66.11% | test acc: 68.37%
Epoch 33/50 | loss: 0.9554 | train acc: 66.34% | test acc: 68.91%
Epoch 34/50 | loss: 0.9473 | train acc: 66.57% | test acc: 67.92%
Epoch 35/50 | loss: 0.9436 | train acc: 66.96% | test acc: 68.48%
Epoch 36/50 | loss: 0.9404 | train acc: 66.73% | test acc: 69.32%
Epoch 37/50 | loss: 0.9355 | train acc: 66.91% | test acc: 67.32%
Epoch 38/50 | loss: 0.9386 | train acc: 66.75% | test acc: 68.66%
Epoch 39/50 | loss: 0.9295 | train acc: 67.24% | test acc: 69.59%
Epoch 40/50 | loss: 0.9304 | train acc: 67.17% | test acc: 68.85%
Epoch 41/50 | loss: 0.9300 | train acc: 67.25% | test acc: 67.34%
Epoch 42/50 | loss: 0.9236 | train acc: 67.69% | test acc: 69.18%
Epoch 43/50 | loss: 0.9222 | train acc: 67.50% | test acc: 69.44%
Epoch 44/50 | loss: 0.9167 | train acc: 67.71% | test acc: 69.17%
Epoch 45/50 | loss: 0.9161 | train acc: 67.73% | test acc: 69.90%
Epoch 46/50 | loss: 0.9123 | train acc: 67.76% | test acc: 68.35%
Epoch 47/50 | loss: 0.9102 | train acc: 67.98% | test acc: 69.61%
Epoch 48/50 | loss: 0.9126 | train acc: 67.77% | test acc: 68.88%
Epoch 49/50 | loss: 0.9070 | train acc: 67.95% | test acc: 69.74%
Epoch 50/50 | loss: 0.9009 | train acc: 68.41% | test acc: 69.31%
Finished Training
```

```
plot_results(train_losses_7, train_accs_7, test_accs_7, num_epochs=50)
```



The results here look very similar to that of problem 4, with the only difference being the learning rate adjusted from lr=0.001 to 0.01, along with the obvious data augmentation bit that adds a length 4 padding on each side to permit a random crop, along with a random horizontal flip. The results show similar loss pattern and slightly higher test performance than the training set. The data augmentation prevents overfitting and suplements sample size.

Problem 8

Change the loss function from Cross Entropy to Mean Squared Error and report the effect.

```
import torch.nn.functional as F

class MSEOneHot(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.mse = nn.MSELoss()
        self.num_classes = num_classes

    def forward(self, outputs, labels):
        targets = F.one_hot(labels, self.num_classes).float()
        return self.mse(outputs, targets)

criterion = MSEOneHot()
```

```
net_4 = Net_4().to(device)
optimizer_4 = optim.SGD(net_4.parameters(), lr=0.01, momentum=0.9)

train_losses_8, train_accs_8, test_accs_8 = run_cnn(net=net_4, optimizer=optimizer_4, num_epochs=50)
```

```
Epoch 1/50 | loss: 0.0905 | train acc: 14.84% | test acc: 19.57%
Epoch 2/50 | loss: 0.0877 | train acc: 18.64% | test acc: 20.37%
Epoch 3/50 | loss: 0.0860 | train acc: 20.53% | test acc: 23.55%
Epoch 4/50 | loss: 0.0852 | train acc: 22.38% | test acc: 25.55%
Epoch 5/50 | loss: 0.0843 | train acc: 24.46% | test acc: 28.38%
Epoch 6/50 | loss: 0.0830 | train acc: 27.17% | test acc: 30.46%
Epoch 7/50 | loss: 0.0817 | train acc: 29.32% | test acc: 33.21%
Epoch 8/50 | loss: 0.0806 | train acc: 31.40% | test acc: 35.78%
Epoch 9/50 | loss: 0.0798 | train acc: 33.44% | test acc: 37.36%
Epoch 10/50 | loss: 0.0791 | train acc: 34.83% | test acc: 39.28%
Epoch 11/50 | loss: 0.0783 | train acc: 36.03% | test acc: 40.00%
Epoch 12/50 | loss: 0.0775 | train acc: 37.08% | test acc: 41.19%
Epoch 13/50 | loss: 0.0769 | train acc: 37.74% | test acc: 41.55%
Epoch 14/50 | loss: 0.0763 | train acc: 38.55% | test acc: 42.45%
Epoch 15/50 | loss: 0.0757 | train acc: 39.10% | test acc: 43.58%
Epoch 16/50 | loss: 0.0751 | train acc: 39.80% | test acc: 44.12%
Epoch 17/50 | loss: 0.0746 | train acc: 40.51% | test acc: 44.68%
Epoch 18/50 | loss: 0.0741 | train acc: 40.95% | test acc: 45.19%
Epoch 19/50 | loss: 0.0734 | train acc: 41.79% | test acc: 45.62%
Epoch 20/50 | loss: 0.0729 | train acc: 42.50% | test acc: 46.32%
Epoch 21/50 | loss: 0.0725 | train acc: 42.97% | test acc: 46.78%
Epoch 22/50 | loss: 0.0720 | train acc: 43.43% | test acc: 46.75%
Epoch 23/50 | loss: 0.0715 | train acc: 44.02% | test acc: 47.42%
Epoch 24/50 | loss: 0.0710 | train acc: 44.76% | test acc: 48.24%
Epoch 25/50 | loss: 0.0706 | train acc: 45.04% | test acc: 48.74%
Epoch 26/50 | loss: 0.0701 | train acc: 45.56% | test acc: 48.89%
Epoch 27/50 | loss: 0.0698 | train acc: 45.77% | test acc: 49.34%
Epoch 28/50 | loss: 0.0694 | train acc: 46.46% | test acc: 50.30%
Epoch 29/50 | loss: 0.0690 | train acc: 46.75% | test acc: 50.80%
Epoch 30/50 | loss: 0.0685 | train acc: 47.45% | test acc: 51.11%
Epoch 31/50 | loss: 0.0682 | train acc: 47.74% | test acc: 50.95%
Epoch 32/50 | loss: 0.0678 | train acc: 48.01% | test acc: 51.50%
Epoch 33/50 | loss: 0.0673 | train acc: 48.81% | test acc: 52.62%
Epoch 34/50 | loss: 0.0670 | train acc: 49.13% | test acc: 52.46%
Epoch 35/50 | loss: 0.0665 | train acc: 49.54% | test acc: 52.83%
Epoch 36/50 | loss: 0.0662 | train acc: 49.66% | test acc: 53.09%
Epoch 37/50 | loss: 0.0659 | train acc: 50.11% | test acc: 53.69%
Epoch 38/50 | loss: 0.0655 | train acc: 50.56% | test acc: 53.68%
Epoch 39/50 | loss: 0.0651 | train acc: 50.98% | test acc: 54.40%
Epoch 40/50 | loss: 0.0648 | train acc: 51.40% | test acc: 54.35%
Epoch 41/50 | loss: 0.0644 | train acc: 51.38% | test acc: 55.10%
Epoch 42/50 | loss: 0.0642 | train acc: 51.79% | test acc: 54.56%
Epoch 43/50 | loss: 0.0638 | train acc: 52.09% | test acc: 55.45%
Epoch 44/50 | loss: 0.0635 | train acc: 52.54% | test acc: 55.99%
Epoch 45/50 | loss: 0.0633 | train acc: 52.56% | test acc: 56.10%
Epoch 46/50 | loss: 0.0629 | train acc: 52.81% | test acc: 56.11%
Epoch 47/50 | loss: 0.0625 | train acc: 53.34% | test acc: 56.69%
Epoch 48/50 | loss: 0.0622 | train acc: 53.36% | test acc: 56.41%
Epoch 49/50 | loss: 0.0619 | train acc: 53.70% | test acc: 57.33%
Epoch 50/50 | loss: 0.0618 | train acc: 54.06% | test acc: 57.22%
Finished Training
```

```
epochs = range(1, 51)
fig, ax = plt.subplots(1, 3, figsize=(16, 4))

ax[0].plot(epochs, train_losses_7, label='Cross entropy')
ax[0].plot(epochs, train_losses_8, label='MSE')
ax[0].set_title('Training loss (different scales!)')

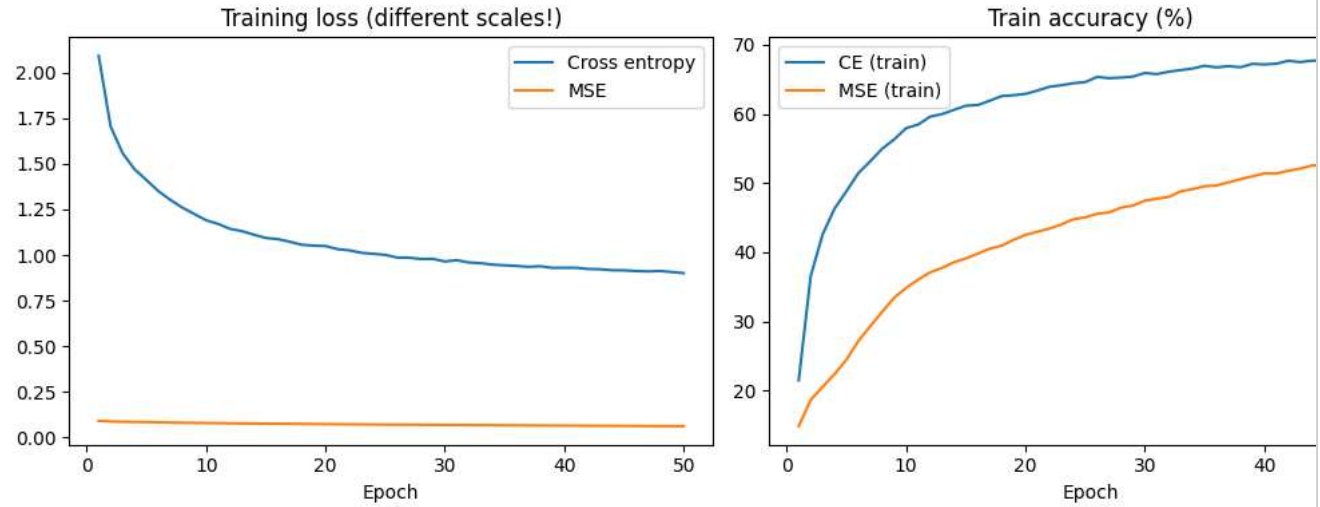
ax[1].plot(epochs, train_accs_7, label='CE (train)')
ax[1].plot(epochs, train_accs_8, label='MSE (train)')
```

```
    ax[1].set_title('Train accuracy (%)')

    ax[2].plot(epochs, test_accs_7, label='CE (test)')
    ax[2].plot(epochs, test_accs_8, label='MSE (test)')
    ax[2].set_title('Test accuracy (%)')

    for a in ax:
        a.set_xlabel('Epoch'); a.legend()
    plt.tight_layout()
    plt.show()

    print(f'CE  | final train: {train_accs_7[-1]:.2f}% | final test: {test_accs_7[-1]:.2f}%')
    print(f'MSE | final train: {train_accs_8[-1]:.2f}% | final test: {test_accs_8[-1]:.2f}%')
```




```
CE  | final train: 68.41% | final test: 69.31%
MSE | final train: 54.06% | final test: 57.22%
```

The train and test accuracy plots clearly expose MSE as an unsuitable loss funcion when the response variable is discrete (car, truck, frog etc). It is essentially the 2-norm of (predicted - true)^2. MSE imposes a slower learning rate in gradient descent and forfeits a lower accuracy ceiling.

Cross-entropy penalizes loss on a different scale, particularly: loss = -log(p_correct), which gives gradient descent the push it needs to flourish.